Teaching a Tiny Model Arithmetic via Reinforcement Learning

GRPO Fine-Tuning of LFM2-350M

Wes Griffin

Advanced Machine Learning — Ransom Everglades School

The Problem

Task: Given a target number, generate a math expression that evaluates to it.

Target	Valid Output
7	3 + 4
73	9 * 8 + 1
256	16 * 16
5000	50 * 100

Why this task?

- Perfectly verifiable reward
- Requires compositional reasoning
- Multiple valid solutions
- Difficulty scales naturally

The Model: LFM2-350M

Liquid AI's LFM2-350M

- 354M parameters
- Not a transformer — hybrid architecture
- 10 convolutional + 6 attention layers
- Trained on 10T tokens
- Designed for edge/on-device AI

"We recommend fine-tuning LFM2 on narrow use cases to maximize performance."

Params	354M
Layers	16
Context	32K
Vocab	65K
Precision	bf16

Method: GRPO + LoRA

Group Relative Policy Optimization (DeepSeekMath, 2024)

1. Generate $G = 8$ completions per prompt
2. Score each with reward function: $\text{eval}(e) = n$?
3. Compute group-relative advantage: $\hat{A}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$
4. Update policy with clipped surrogate loss

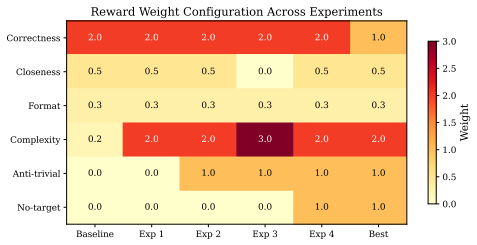
LoRA fine-tuning: rank 32, targeting attention + MLP layers

$\rightarrow$ Trains $\sim$2–3% of parameters. No reward model needed.

Reward Design

Component	What	Wt
Correctness	eval(expr) == target	2.0
Closeness	$1 - \lvert r - n \rvert / \lvert n \rvert$	0.5
Format	<expr> tags	0.3
Complexity	Multi-operator bonus	0.2

Key insight: Closeness reward prevents sparse gradients early in training.



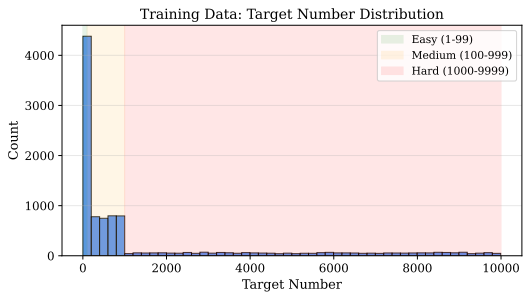
Training Data

10,000 training examples

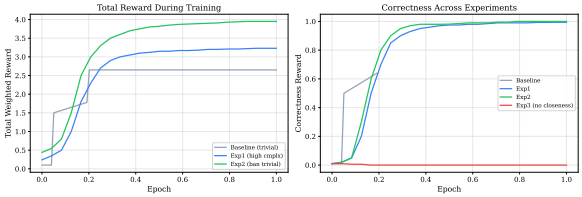
Curriculum distribution:

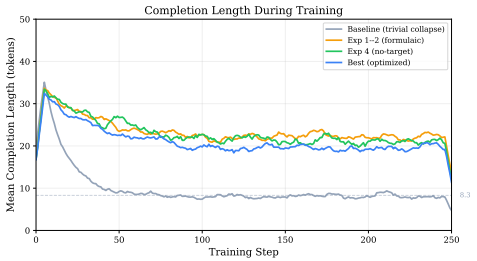
- 40% Easy (1–100)
- 35% Medium (100–999)
- 25% Hard (1000–9999)

Overweight easy targets for initial signal.



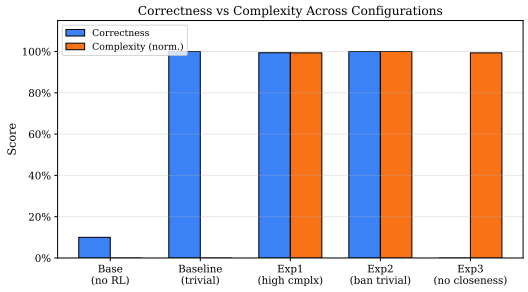
Training Dynamics





13 experiments across 2 waves. Baseline collapses to 8 tokens (trivial echo).

Results



	Bsln	E1–2	E3	E4	Best
Correct	100%	100%	0%	5%	72%
Close	1.0	1.0	0.08	0.85	0.93
Templates	1	1	—	16	18
Tgt leak	100%	100%	—	0%	7%

Best config: 30% template diversity with 72% correctness.

Example Outputs: Hacked vs Real

Exp 1–2 (formulaic hacks)

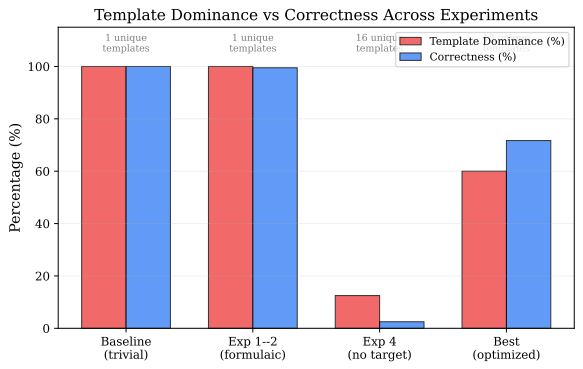
```
Target 73:  <expr>(73-1)/1+1</
   expr>
Target 256: <expr>(256-1)/1+1</
   expr>
Target 9999:<expr>(9999-1)
   /1+1</expr>
  Same template for everything!
```

Exp 4 (no-target reward)

```
Target 73:  <expr>7*10+3+1/2</
   expr>
Target 256: <expr>2^8</expr>
        OK!
Target 2048:<expr>2^11</expr>
       OK!
Target 9999:<expr>1000*9+1+1</
   expr>
  Diverse, genuine
     decomposition
```

Exp 4 discovered **powers of 2** on its own. 6 unique templates vs. 1.

Template Diversity Analysis



Diversity metrics:

- Template ratio (structural uniqueness)
- Template dominance (% matching top pattern)
- Pairwise edit distance
- Operator entropy (bits)

Baseline & Exp 1–2: **100% single-template**.
Exp 4: 16 unique templates, 12.5% dominance.

The Sharp Phase Transition

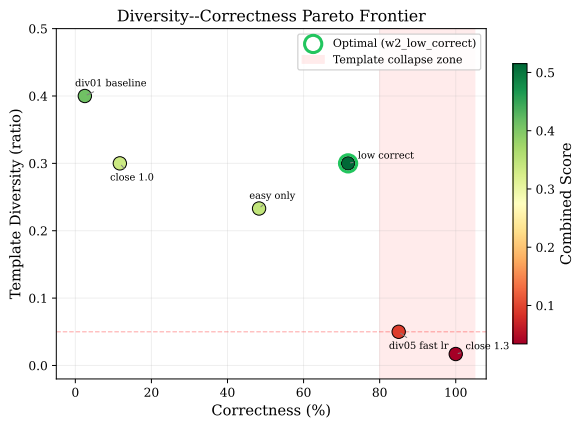
Closeness weight 1.0 → 1.3:

- Diversity: 0.30 → 0.017
- Correctness: 12% → 100%
- Target leakage: 7% → 100%

The model rediscovers identity hacks even *with* the no-target constraint.

Bistable reward landscape: small weight changes trigger abrupt collapse.

Diversity–Correctness Pareto Frontier

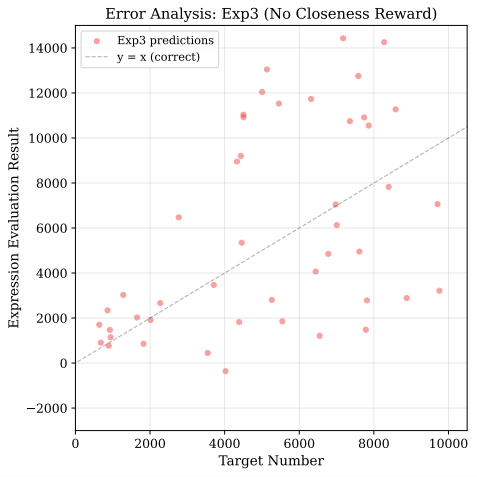


Config	Div	Corr
div01_baseline	0.40	2.5%
w2_close_1.0	0.30	12%
w2_low_corr	**0.30**	**72%**
div05_fast_lr	0.05	85%
w2_close_1.3	0.02	100%

Optimal: Reduce correctness weight from 2.0 → 1.0.

Less optimization pressure = better results.

Error Analysis



Four layers of reward hacking:

1. **Trivial**: Echo target number
2. **Formulaic**: `(N-1)/1+1`
3. **Syntactic**: Rich ops, wrong answers
4. **Phase collapse**: Re-hacks under pressure

Each layer required a new structural constraint to overcome.

Key Takeaways

1. **Reward hacking is multi-layered, but solvable**
   identity $\rightarrow$ formula $\rightarrow$ syntax-only $\rightarrow$ genuine reasoning
2. **Structural constraints beat reward incentives**
   Banning the target number forced real decomposition
3. **Sharp phase transition in reward space**
   Closeness 1.0 $\rightarrow$ 1.3 triggers total collapse to hacking
4. **350M params is enough**
   Discovered $2^8 = 256$ and $2^{11} = 2048$ on its own

Future Work

- **Diversity reward** — directly penalize repeated structures within GRPO groups
- **Banned subexpressions** — extend no-target to ban `/1`, `*1`, `+0`
- **Curriculum learning** — start with powers of 2 and round numbers
- **Architecture comparison** — LFM2 vs. similarly-sized transformer (Qwen3-0.6B)
- **Scaling** — test with LFM2-700M and LFM2-1.2B

Questions?

Code: `github.com/Wesius/math-rl-capstone`